

Smart Photo Drive

Project Proposal for UCS503 – Software Engineering
Submitted to: Ms. Nisha Thakur
Thapar Institute of Engineering and Technology

Harshjyot Kaur – 1024031176 Jaiveer Singh – 1024030976
Archita Sharma – 1024030715 Avneet Kaur – 1024031173

August 19, 2026

1 Higher-order goal

... secondary framing

The goal of **Smart Photo Drive** is to simplify the process of finding personal photographs from large collections of event photographs. Events such as college functions, conferences, festivals, and other gatherings can generate hundreds or thousands of photographs, making manual searching inconvenient and time-consuming.

The immediate engineering objective is to develop a web-based system that uses computer vision to identify photographs containing a particular user and present those photographs through an easy-to-use interface.

The project focuses on combining an AI-assisted image retrieval workflow with practical software engineering principles such as modular design, authentication, database management, testing, scalability, and deployment.

2 Time-to-value

... primary heuristic

- We will first develop the basic end-to-end workflow in which event photographs can be uploaded and a user can search for photographs containing their face.
- Development will be carried out incrementally, beginning with a working prototype and then improving recognition quality, usability, performance, security, and deployment.
- Early iterations will prioritize a functional workflow so that the core idea can be tested using a controlled collection of event photographs.
- Feedback obtained during development and testing will be used to refine both the user interface and the photo retrieval process.

3 Problem Statement

Large events often result in a very large number of photographs being captured by photographers. After the event, participants usually have to manually browse through these photographs to find the images in which they appear.

Common problems include:

- **Manual searching:** Users may need to inspect a large number of photographs individually.

- **Time consumption:** Finding a specific person’s photographs becomes increasingly difficult as the collection grows.
- **Lack of person-based retrieval:** Conventional photograph folders are generally organized by event, date, or filename rather than by the people appearing in the photographs.
- **Repeated effort:** Multiple participants may independently perform the same manual search.
- **Large photo collections:** Event photographers and organizers need an efficient way to manage and distribute large numbers of photographs.

Why this matters now: The increasing use of digital photography has made it easy to generate large event collections. However, finding an individual’s photographs within these collections still requires significant manual effort.

4 Proposed Solution

... Software Proposition

4.1 Overview

Smart Photo Drive will be developed as a **web-based AI-assisted photo retrieval system**. An event photographer or event manager will be able to upload photographs from an event, while users will be able to provide a reference photograph containing their face.

The system will process the event photographs and identify photographs that are likely to contain the queried user.

The proposed system will contain the following major components:

- **Event management:** Event managers can create events and upload the corresponding photograph collection.
- **User photo search:** A user can provide a reference photograph to initiate the retrieval process.
- **Face recognition:** Computer vision techniques will be used to identify and compare faces present in photographs.
- **Photo retrieval:** Photographs containing matching faces will be identified and returned to the user.
- **Photo viewing:** Users will be able to view the photographs retrieved for them and access permitted downloads.
- **Role-based access:** Different workflows will be provided for normal users and event managers.

4.2 Core workflow

1. An event manager creates an event and uploads the event photographs.
2. The system processes the uploaded photographs and detects faces present in the images.
3. A user registers or logs in and provides a reference photograph containing their face.
4. The system extracts relevant facial information from the reference photograph.
5. The reference information is compared with faces identified in the event photographs.
6. Photographs satisfying the selected matching criteria are retrieved.
7. The matched photographs are displayed to the user through the web interface.

4.3 Operational constraints

- The system should remain usable on commonly available desktop and mobile devices.
- The initial implementation will focus on manageable event collections while keeping the architecture extensible.
- The recognition process should provide a reasonable balance between accuracy and computational requirements.
- User photographs and event resources should be protected through appropriate authentication and access controls.
- Only information required for the application's operation should be collected and stored.

5 Solution Approach

... Engineering focus

This is an engineering course project, so the focus will be on integrating established technologies into a complete software system rather than developing a new face-recognition algorithm.

5.1 Face recognition and image processing

The project will investigate established computer vision techniques for detecting and recognizing faces.

OpenCV will be considered for image processing and supporting computer vision operations. The `face_recognition` library, which is based on `dlib`, will be evaluated for generating facial encodings and performing face comparisons.

The exact recognition configuration and matching threshold will be determined during implementation and testing using representative event photographs.

Factors such as lighting, image quality, pose, facial orientation, and partial occlusion will be considered during evaluation.

5.2 Web architecture

... proposed technology stack

The proposed system will follow a modular web architecture consisting of a frontend, backend, database, image storage, and AI-processing components.

- **Frontend:** React will be used for developing the interactive web interface. Vite will be used as the frontend development and build tool, while Tailwind CSS will be considered for responsive interface styling.
- **Backend:** Flask will be used to develop the backend APIs and application logic required for authentication, event management, photo processing, and retrieval.
- **Database:** MongoDB will be used for storing application information such as users, events, photograph metadata, and processing information.
- **Image storage:** Cloud-based object or image storage will be evaluated for storing large photograph collections separately from application metadata.
- **AI processing:** The face recognition and image processing components will interact with the backend to process photographs and provide retrieval results.

5.3 Authentication and access control

The system will provide separate workflows for its primary roles:

- **Users:** Users will be able to register, authenticate, search for photographs, and access photographs associated with permitted events.
- **Event Managers:** Event managers will be able to create events, upload photographs, and manage event-related resources.

Token-based authentication such as JSON Web Tokens (JWT) will be considered for secure authentication. Passwords will be protected using an appropriate password-hashing mechanism rather than being stored directly.

5.4 Development and deployment

The project will use GitHub for version control and collaborative development.

The application will be designed so that the frontend and backend can be deployed independently. Docker will be considered for containerizing backend services and creating a consistent development and deployment environment.

The frontend can be deployed using a modern cloud hosting platform, while the backend can be deployed using a suitable container-hosting service.

5.5 CI/CD and development automation

GitHub Actions will be considered for automating repetitive software engineering tasks during development.

Potential automated tasks include:

- running backend tests,
- validating the frontend build,
- checking code changes,
- building deployment artifacts, and
- supporting repeatable deployment processes.

The exact CI/CD workflow will be finalized as the project progresses.

6 Evaluation Criterion

... measurable and attributable

The system will be evaluated using metrics related to recognition quality, retrieval performance, and usability.

6.1 Primary evaluation metric

Photo retrieval precision: the proportion of photographs returned by the system that actually contain the queried user.

A manually verified test collection will be used to compare the system's retrieved results with the expected photographs.

6.2 Secondary evaluation metrics

- **Recall:** percentage of relevant photographs successfully retrieved.
- **False-match rate:** percentage of retrieved photographs that do not contain the target user.
- **Retrieval time:** time required to return matching photographs after a search request.
- **Processing time:** time required to process a collection of event photographs.
- **User experience:** ease with which users can upload a reference photograph and access their results.
- **System reliability:** successful completion of authentication, upload, processing, search, and retrieval operations during testing.

6.3 Pilot validation plan

- Prepare a representative collection of event photographs containing multiple individuals.
- Select several test users and provide reference photographs for them.
- Compare retrieved photographs with manually verified photographs.
- Measure retrieval and processing time for different collection sizes.
- Collect basic feedback from users regarding the simplicity and usefulness of the retrieval process.

7 Scalability

...with foresight

The proposed architecture should remain capable of handling increasing numbers of users, events, and photographs.

1. Scalable architecture:

- Separate frontend and backend responsibilities.
- Store large image files separately from application metadata.
- Process photographs in batches where appropriate.
- Store useful processing results to avoid unnecessary repeated computation.
- Use pagination and efficient database queries for large collections.

2. Deployment scalability:

- Use cloud-based infrastructure where required.
- Containerize backend components for consistent deployment.
- Keep storage and processing components modular so they can be scaled independently if required.

8 Engine availability heuristic

The project can be implemented using mature and widely available software technologies. The initial technology direction includes:

- **React, Vite, and Tailwind CSS** for the frontend.
- **Flask** for backend APIs and application logic.
- **MongoDB** for application and event metadata.
- **OpenCV and face_recognition/dlib** for computer vision and face recognition.
- **JWT-based authentication** and secure password hashing for access control.
- **Cloud or object storage** for large photograph collections.
- **Docker** for reproducible deployment environments.
- **GitHub Actions** for development automation and continuous integration.

These technologies provide a practical starting point, while implementation details and alternative tools will be evaluated during subsequent development iterations.

9 Project scope and deliverables

... iteration-friendly

9.1 Initial deliverable

... first iteration

The first implementation iteration will focus on establishing the basic end-to-end workflow:

- Basic React and Vite frontend.
- Initial responsive interface using Tailwind CSS.
- Event creation and photograph upload.
- User reference photograph upload.
- Flask backend APIs.
- MongoDB integration.
- Basic image processing.
- Initial face detection and recognition pipeline.
- Retrieval and display of matched photographs.
- Basic authentication for users and event managers.

9.2 Subsequent deliverables

- Improvement of recognition accuracy.
- Improved user and event-manager workflows.
- Cloud-based photograph storage.
- Improved handling of larger photo collections.
- Photo viewing and downloading.
- Search, filtering, and event management improvements.
- Automated backend and frontend testing.
- Docker-based deployment.
- GitHub Actions-based continuous integration.
- Deployment of the complete web application.

10 Risks and mitigations

- **Face recognition accuracy:** Differences in lighting, pose, image quality, and occlusion may result in incorrect or missed matches. Testing with representative photographs and threshold evaluation will be used to improve reliability.
- **Large photo collections:** Processing a large number of photographs may require significant computational resources. Batch processing and efficient storage of processing results will be investigated.
- **Privacy:** Event photographs and facial information require appropriate protection. Authentication and access control will be implemented to restrict access to authorized resources.
- **Storage requirements:** Large event collections may require significant storage capacity. Cloud or object storage solutions will be evaluated according to project requirements.
- **Security:** Incorrect authentication or authorization could expose private photographs. User and event-manager permissions will therefore be tested separately.
- **Deployment complexity:** Integration of the frontend, backend, database, AI processing, and storage components may introduce deployment challenges. Containerization and automated testing will be introduced incrementally.
- **Evaluation ambiguity:** Recognition quality will be evaluated using a manually verified test collection and measurable metrics such as precision, recall, and false-match rate.

11 Summary

This project proposes **Smart Photo Drive**, a web-based AI-assisted system for automatically finding photographs of a user from large collections of event photographs.

The proposed workflow allows an event manager to upload an event photo collection while users provide a reference photograph containing their face. Computer vision techniques are then used to identify likely matching photographs, which are presented through the web interface.

The initial technology direction includes React, Vite, and Tailwind CSS for the frontend, Flask for the backend, MongoDB for application data, and established computer vision libraries for face processing. Cloud-based storage, authentication, containerization, and automated development workflows will be incorporated as the project progresses.

The project emphasizes software engineering principles including incremental development, modular architecture, authentication, testing, measurable evaluation, scalability, version control, and deployment.

The final objective is to replace the manual process of searching through large event photo collections with a faster, automated, and user-friendly photo retrieval experience.